Industr.IA

E-book

# IA agêntica:

## A decisão Workflow vs Agente e os Modos de falha



Curso base de IA | Unidade M2

Prof.: Arthur Nascimento

# SUMÁRIO


1. A decisão entre workflow e agente 3
2. Modos de falha e a centralidade do raciocínio 6
3. Fechamento decisório: a autonomia no ciclo do projeto 10
4. Conclusão 13

## Unidade M2

# 1. A decisão entre workflow e agente: A heurística do branching dinâmico

Esta seção pertence à camada decisória do módulo. Ao final dela, você será capaz de aplicar a heurística workflow versus agente a um cenário industrial, justificando a escolha em termos de branching dinâmico, custo e transparência. A pergunta que organiza a seção não é qual framework adotar, e sim uma anterior, que costuma ficar implícita nas discussões de squad: vale a pena entregar ao próprio sistema a decisão de qual caminho seguir? É essa decisão, e não a escolha de ferramenta, que define a lente da autonomia.

As duas arquiteturas tratadas no e-book anterior se distinguem por quem controla o caminho. No workflow determinista, quem desenha a solução fixa de antemão a sequência de passos e as ramificações possíveis. No agente, o sistema decide, a cada passo, o que fazer em seguida, em função do que acabou de observar. Vale insistir num ponto que costuma gerar confusão: a diferença não está na presença de um modelo. Um workflow pode chamar um modelo dentro de um passo, para classificar um texto ou extrair campos, sem deixar de ser um workflow. O que muda é o controle do caminho, fixo no workflow e decidido em execução no agente. Essa é a decisão de arquitetura mais consequente do módulo, porque dela dependem o custo, a previsibilidade e a auditabilidade de tudo o que vem depois.

O conceito que separa um caso do outro é o branching dinâmico. Há branching dinâmico quando o próximo passo depende de resultados intermediários que não se consegue enumerar de antemão. Um teste prático ajuda a decidir. Tente desenhar o fluxograma completo da solução antes de executá-la, com todas as ramificações nomeadas. Se você consegue, com um número administrável de ramos, o caminho é fixável e o caso é de workflow. Se as ramificações dependem de conteúdo que só aparece em tempo de execução, e são numerosas ou imprevisíveis demais para enumerar, o caminho é dinâmico, e só então o agente entra em consideração.

GLOSSÁRIO

**Branching dinâmico**

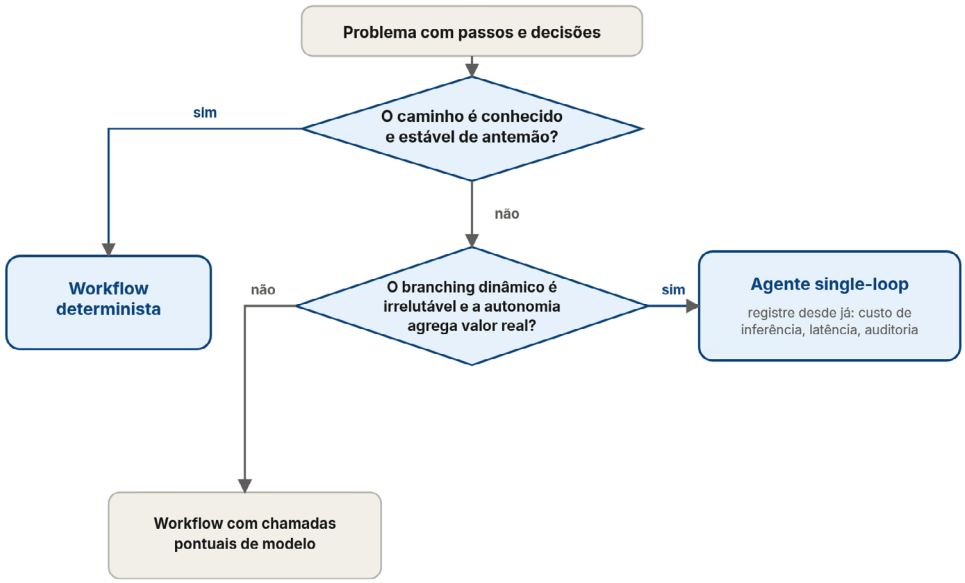
Situação em que o próximo passo de uma solução depende de resultados que só surgem durante a execução, de modo que a sequência de ações não pode ser fixada de antemão. É o sinal central que distingue um problema de agente de um problema de workflow.

A heurística de decisão é direta e tem respaldo na prática de engenharia da área: comece pela solução mais simples e aumente a autonomia apenas quando o ganho for claro. Na dúvida, o default é o workflow. As razões são concretas, e cada uma delas se transforma em problema quando se sobe para um agente sem necessidade. O workflow é mais previsível, porque o caminho é conhecido, o que torna a depuração simples: quando algo falha, você sabe em qual passo olhar. Custa menos, porque faz menos chamadas ao modelo, e o custo de um agente cresce com o número de passos. E é mais simples de testar, porque se comporta de forma determinista, enquanto um agente pode seguir caminhos diferentes para a mesma entrada.

Sobe-se para o agente quando duas condições se encontram, e não apenas uma. A primeira é que o branching dinâmico seja irrelutável, o que significa que você tentou fixar o fluxo e não conseguiu sem enumerar um número impraticável de casos. A segunda é que a autonomia agregue valor real, o que significa que a flexibilidade resolve um problema concreto da tarefa, e não que ela soa mais avançada numa apresentação. Quando apenas uma das condições está presente, a resposta em geral ainda é o workflow. Um problema com muitas ramificações, mas todas conhecidas, é um workflow grande, não um agente. Um problema com autonomia desejável, mas caminho fixável, também é um workflow.

Existe um caminho do meio que resolve boa parte dos casos que à primeira vista parecem pedir um agente: o workflow com chamadas pontuais de modelo. Muitos problemas industriais se resolvem com um pipeline fixo em que um ou dois passos chamam um modelo para uma tarefa bem delimitada, como classificar um documento, extrair informações de um texto livre ou redigir um resumo padronizado. O caminho permanece fixo e auditável, e o modelo entra apenas onde a interpretação de linguagem é necessária. Isso não é um agente, porque o sistema não decide a sequência. Reconhecer esse meio-termo desfaz a falsa dicotomia entre um script rígido e um agente totalmente autônomo, que é uma das principais fontes de over-engineering na área.

Figura 1 - Heurística de decisão entre workflow e agente




Fonte: autor.

Fluxograma intitulado “Figura 1
– Heurística de decisão entre workflow e agente”, com o subtítulo “Workflow por padrão, agente por exceção” e a observação “a heurística do branching dinâmico”. No topo, um retângulo com a frase “Problema com passos e decisões” conduz a um losango com a pergunta “O caminho é conhecido e estável de antemão?”. Se a resposta for “sim”, uma seta segue para a esquerda até um retângulo com o texto “Workflow determinista”. Se a resposta for “não”, a seta desce até um segundo losango com a pergunta “O branching dinâmico é irrelevante e a autonomia agrega valor real?”. A partir desse ponto, se a resposta for “sim”, uma seta segue para a direita até um retângulo com o texto “Agente single-loop”, acompanhado da observação em letras menores: “registre desde já: custo de inferência, latência, auditoria”. Se a resposta for “não”, a seta desce até um retângulo com o texto “Workflow com chamadas pontuais de modelo”. Na parte inferior da figura, uma recomendação resume a heurística: “Na dúvida, comece pelo workflow. Suba para agente apenas quando o ganho compensar o custo.” Abaixo da imagem aparece a legenda “Fonte: autor.”

A decisão se organiza em três eixos, que o template T2.1 vai formalizar e que vale antecipar aqui. O primeiro é o branching dinâmico: o caminho é fixável ou não? O segundo é o custo: quantas chamadas a mais o agente introduz em relação a um workflow equivalente, e esse acréscimo é sustentável no volume previsto? O terceiro é a transparência: você consegue auditar e explicar a sequência de decisões a um stakeholder ou a um regulador? Os três eixos são lidos em conjunto, e não isoladamente.

O ponto importante é que cada eixo pode vetar a escolha do agente, mesmo quando os outros a favorecem. Um problema pode ter branching alto e, ainda assim, ser resolvido por um fluxo mais rígido porque a transparência é um requisito regulatório que a autonomia comprometeria. Outro pode comportar um agente do ponto de vista técnico, mas ter um custo de inferência incompatível com o volume diário esperado. A leitura conjunta dos eixos é o que diferencia uma decisão defensável de uma preferência estética por arquitetura, e é exatamente o que se espera registrar no instrumento decisório do módulo.

### Exemplo aplicado

Numa indústria de equipamentos rotativos, a triagem de chamados de manutenção recebe textos livres de operadores. Uma versão inicial usava um workflow: classificar o chamado, consultar a tabela de severidade e encaminhar. Funcionava para a maioria. A equipe avaliou trocar por um agente porque alguns chamados exigiam consultar histórico, manuais e ordens abertas em ordem variável. A decisão final manteve o workflow para os casos comuns e reservou o agente apenas para a fila de exceções, onde o branching era de fato imprevisível.

Vale conectar essa decisão ao que você praticou no notebook. Lá, os passos de cada tarefa poderiam, em princípio, ser fixados de antemão. Foi a tarefa B que expôs o ponto: o agente tentou calcular, observou um erro e mudou de rumo. Essa variação de caminho, decidida em execução, é exatamente o tipo de branching dinâmico que justifica um agente quando é recorrente e valioso. Se o caminho da tarefa B fosse sempre o mesmo, um workflow o resolveria com menos custo e mais previsibilidade. A pergunta reflexiva que fechou o notebook se resolve aqui: a variação de caminho justifica um agente apenas quando não se consegue fixá-la sem perder aquilo que ela traz de útil. Quando a variação é aparente, mas o caminho real é sempre o mesmo, o agente é custo sem retorno.

**ATENÇÃO**

A confusão decisória mais frequente é adotar o agente como default moderno, por ser o que aparece nas demonstrações, e só depois procurar uma justificativa. O efeito é uma solução mais cara, mais lenta e mais difícil de auditar do que o problema exigia. A heurística inverte o ônus da prova: quem precisa se justificar é a autonomia, não o workflow.

**SAIBA MAIS**

ANTHROPIC. Building Effective Agents. Anthropic Engineering, 2024. Disponível em https://anthropic.com/engineering/building-effective-agents. A comparação instrumentada entre frameworks como LangGraph, AutoGen e CrewAI é tratada no curso de LLMs da Fase 2 do programa.

A heurística, em resumo, desloca o esforço de decisão para antes da implementação. Escolher entre workflow e agente é uma decisão de arquitetura com consequências de custo e de auditoria, e não um detalhe que se acerta depois de codificar. A próxima seção trata do que a escolha do agente introduz de novo: os modos de falha próprios de um sistema que decide em tempo de execução, e as mitigações que precisam ser desenhadas junto com a solução, e não acrescentadas quando o problema já apareceu em produção.

## 2. Modos de falha e a centralidade do raciocínio

Esta seção pertence à camada decisória. Ao final dela, você será capaz de enumerar os modos de falha recorrentes de sistemas agênticos e mapear mitigações, e de decidir onde colocar o raciocínio, em um modelo de linguagem ou em um orquestrador baseado em regras. O ponto de partida é reconhecer que esses modos de falha não são casos raros de borda, que se resolvem com um modelo melhor. Eles são estruturais a qualquer arquitetura que decide o caminho em tempo de execução, e por isso precisam entrar na conta da decisão tomada na seção anterior.

Um sistema que decide a cada passo tem uma superfície de falha maior que um workflow de caminho fixo, por duas razões cumulativas. A primeira é que cada decisão de runtime pode errar, e há muitas decisões. A segunda é que os erros se encadeiam, por-

que a saída de um passo é a entrada do seguinte. Quatro modos de falha aparecem com frequência em PD&I industrial e merecem nome próprio, aqui apresentados na forma em que se manifestam no laço de um agente.

Os dois primeiros modos dizem respeito à decisão em si. O loop ocorre quando o agente repete ações sem progredir em direção ao objetivo, por exemplo pedindo a mesma busca várias vezes ou alternando entre duas ações sem nunca convergir. A seleção de ferramenta errada ocorre quando o agente escolhe a tool inadequada para o que precisa, ou a tool certa com argumentos errados. Os dois consomem chamadas pagas e tempo, e os dois só se tornam visíveis quando há um trace para inspecionar, porque do lado de fora o sistema apenas parece lento ou parece dar uma resposta estranha, sem explicação aparente.

Os dois modos seguintes dizem respeito à propagação. A injeção de prompt (prompt injection), aqui tratada no cenário que importa para agentes, ocorre quando conteúdo vindo do ambiente, como uma observação de uma tool ou um documento recuperado, contém instruções que desviam o agente do seu objetivo original. É uma falha de segurança específica de sistemas que combinam a leitura de texto não confiável com a capacidade de executar ações, e não tem paralelo direto num workflow fechado. O cascading error ocorre quando um erro inicial vira entrada do passo seguinte e se amplifica ao longo do laço, de modo que uma única observação ruim contamina todas as decisões posteriores, sem que o agente perceba que partiu de uma base errada.

GLOSSÁRIO

**prompt injection**

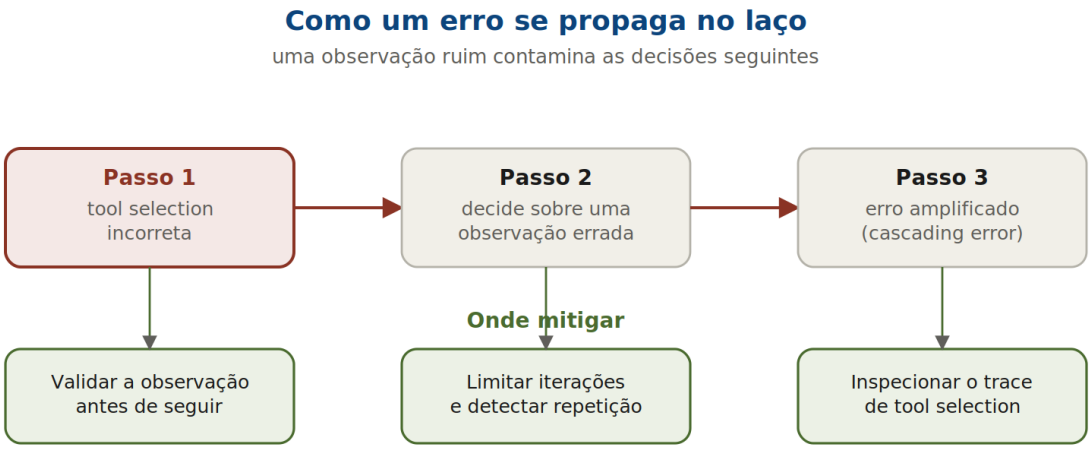
Ataque ou falha em que conteúdo lido pelo agente a partir do ambiente, como um documento ou uma observação de tool, contém instruções que o desviam do objetivo original. É um risco próprio de sistemas que combinam leitura de texto não confiável com a capacidade de executar ações.

Quadro1 - Modos de falha recorrentes e mitigações

| Modo de falha | Como aparece | Mitigação típica |
|---|---|---|
| Loop | Repete ações sem progredir no objetivo | Limitar iterações e detectar repetição de estado |
| Tool selection errada | Escolhe a ferramenta ou os argumentos errados | Restringir o conjunto de tools e validar argumentos |

| | | |
|---|---|---|
| Prompt injection | Conteúdo do ambiente desvia o agente | Isolar e tratar texto não confiável como dado, não instrução |
| Cascading error | Um erro inicial se amplifica no laço | Validar a observação antes da próxima decisão |

Figura 2 - Propagação de erro no laço e pontos de mitigação




Fonte: autor.

Fluxograma intitulado "Figura 2 – Propagação de erro no laço e pontos de mitigação". Abaixo do título, em destaque, aparece a frase "Como um erro se propaga no laço", seguida do subtítulo "uma observação ruim contamina as decisões seguintes". O diagrama apresenta três etapas conectadas por setas da esquerda para a direita. A primeira é um retângulo, identificado como "Passo 1", contendo o texto "tool selection incorreta". Uma seta leva ao "Passo 2", em um retângulo , com o texto "decide sobre uma observação errada". Outra seta conduz ao "Passo 3", também em um retângulo, com o texto "erro amplificado (cascading error)". Abaixo de cada etapa há um retângulo indicando a estratégia de mitigação correspondente. Sob o Passo 1, lê-se "Validar a observação antes de seguir". Entre o Passo 2 e seu bloco de mitigação aparece o destaque "Onde mitigar", indicando o retângulo com o texto "Limitar iterações e detectar repetição". Sob o Passo 3, o retângulo apresenta a orientação "Inspecionar o trace de tool selection". Na parte inferior da figura, uma observação resume a mensagem principal: "A mitigação age entre a observação e a próxima decisão, não depois do resultado final." Abaixo da imagem consta a legenda "Fonte: autor."

O padrão das mitigações é revelador, e é o que torna a lista acima utilizável na prática. Quase todas elas agem no mesmo lugar, entre a observação e a próxima decisão. Validar a observação com checagens simples antes de prosseguir, limitar o número de iterações e detectar repetição de estado, restringir o conjunto de tools disponíveis e os argumentos que cada uma aceita, e tratar todo texto vindo do ambiente como dado, e nunca como instrução a ser obedecida. Nenhuma dessas medidas torna o agente mais inteligente. Todas tornam o sistema ao redor do agente mais robusto, e é nesse sistema, e não no modelo, que se concentra o trabalho de engenharia confiável.

Isso conduz à segunda decisão da seção, a centralidade do raciocínio. A pergunta é onde colocar a decisão aberta, aquela que de fato exige interpretar uma situação nova, e onde manter o controle determinista. A boa prática é restringir o modelo de linguagem aos pontos em que esse tipo de decisão é realmente necessário, e cercá-lo de controle determinista: validação das saídas, limites de iteração, roteamento fixo entre etapas. Quanto mais do caminho puder ser determinista, menor a superfície de falha, mais barato o sistema e mais simples a auditoria. O notebook da etapa passada ilustra esse princípio de forma deliberada: o reasoning engine ali é determinista, justamente para que o laço e o trace fiquem reprodutíveis e legíveis, e para que a atenção recaia sobre a estrutura, e não sobre a sorte de uma geração.

Na prática, traçar essa fronteira é a própria decisão de design. Decisões que exigem interpretar uma entrada nova e ambígua, como entender um chamado escrito em linguagem livre, são candidatas legítimas ao modelo. Decisões que podem ser expressas como regra, como rotear segundo um campo, validar um formato ou somar valores, devem ficar no orquestrador determinista, onde são baratas, testáveis e auditáveis. O erro comum é deixar o modelo decidir também essas últimas, por conveniência de implementação, e herdar o não determinismo e o custo onde eles não traziam benefício algum. Quanto mais nítida a fronteira entre o que o modelo decide e o que o código controla, mais simples fica operar o agente e mais barato fica corrigi-lo quando algo falha.

**Exemplo aplicado**

Uma equipe de engenharia de dados construiu um agente para inferir o schema de arquivos recebidos de fornecedores. A primeira versão deixava o modelo decidir todos os passos, e sofria com loops e seleção de tool errada. A segunda versão fixou o esqueleto do pipeline de forma determinista, ler, amostrar e validar, e reservou o modelo apenas para o passo ambíguo de propor o tipo de cada coluna. O número de chamadas caiu, o trace ficou auditável e as falhas viraram exceções tratáveis.

É útil reler aqui o trace da tarefa B do notebook da etapa passada, porque ele materializa o mecanismo de recuperação. O passo 1 foi uma seleção de ferramenta incorreta: o agente tentou calcular antes de buscar o dado de que precisava. O que tornou o sistema recuperável não foi um modelo melhor, foi a observação de erro devolvida pela ferramenta e a checagem feita antes do passo seguinte, que levou o agente a buscar o dado e só então calcular. Esse é o desenho que transforma uma falha isolada em recuperação, em vez de um cascading error. A pergunta reflexiva do notebook, sobre identificar uma seleção de ferramenta correta e uma incorreta, aponta justamente para esse ponto de validação entre passos, que é onde a robustez de um agente se decide na prática.

A terceira decisão é a tolerância a falhas em loop, que se traduz em três custos a serem aceitos de forma consciente. Há o custo financeiro, porque cada nova tentativa de recuperação é uma chamada paga ao modelo. Há o custo de transparência, porque sem um trace registrado a falha é invisível e não há o que depurar quando o resultado vem errado. E há o custo de auditoria, porque em setor regulado é preciso reconstruir, depois do fato, porque o agente agiu como agiu. Decidir a tolerância a falhas é, na prática, decidir quanto investir em validação e em observabilidade antes de colocar o agente em produção, e esse investimento entra no orçamento do projeto desde o início, não como ajuste posterior.

> ⚠ ATENÇÃO
>
> A confusão decisória aqui é tratar o trace, ou a cadeia de raciocínio que o modelo verbaliza, como prova de que o agente agiu corretamente. O trace mostra o que o agente fez e disse, não que estava certo. Confiar no relato do próprio modelo como garantia de correção dispensa a validação externa justamente onde ela é mais necessária.

Os modos de falha não desqualificam os agentes. Eles definem o custo de operá-los bem, custo que precisa ser conhecido antes de escolher a arquitetura, e não descoberto depois. A próxima seção traduz esse custo, e a decisão de autonomia como um todo, em compromissos explícitos do ciclo do projeto, que precisam estar registrados antes de a solução ser desenhada.

## 3. Fechamento decisório: a autonomia no ciclo do projeto

Esta seção pertence à camada decisória e fecha o módulo. Ao final dela, você será capaz de articular a decisão agente versus workflow ao ciclo de um projeto, traduzindo-a em compromissos de custo de inferência, latência e observabilidade, e de estimar, em ordem de grandeza, o custo adicional de uma arquitetura agentic em relação a uma chamada direta de modelo. O objetivo é que a escolha de autonomia deixe de ser uma decisão técnica isolada e passe a ser tratada como o que ela é, um compromisso de projeto com consequências de cronograma e de orçamento.

A ideia que organiza o fechamento é que a escolha por um agente é um compromisso de produto e de arquitetura assumido no início, e não uma otimização deixada para o fim. Optar pelo agente compromete o projeto com custos que precisam estar na mesa antes do desenho da solução, porque eles mudam a infraestrutura necessária, o cronograma de validação e até a viabilidade econômica da proposta. Tratar esses custos como detalhe de implementação, a ser resolvido depois, é uma das causas mais comuns de soluções agênticas que funcionam na demonstração e não se sustentam em produção.

O primeiro compromisso é o custo de inferência, que é multiplicativo. Um agente com k passos faz ao menos k chamadas ao modelo, e mais ainda quando há tentativas repetidas de recuperação. Uma chamada direta faz uma única inferência. A diferença não é um detalhe percentual: pode ser de uma ordem de grandeza. Estimar o custo pela média, isto é, pelo caminho feliz em que tudo dá certo na primeira tentativa, subestima o sistema, porque o teto é dado pelo pior caso, aquele em que o agente erra, observa o

erro e tenta de novo, às vezes mais de uma vez antes de convergir.

O segundo compromisso é a latência, que é sequencial. O tempo total de resposta soma cada chamada ao modelo e cada execução de tool, na ordem em que acontecem, porque um passo depende do resultado do anterior e não pode ser antecipado. Um agente é, por construção, mais lento que uma chamada única para o mesmo objetivo. Em aplicações com requisito de latência, como uma interface conversacional que precisa responder em tempo real ou um sistema que decide dentro de uma janela curta, essa soma pode vetar o agente, por melhor que ele resolva a tarefa em termos de qualidade.

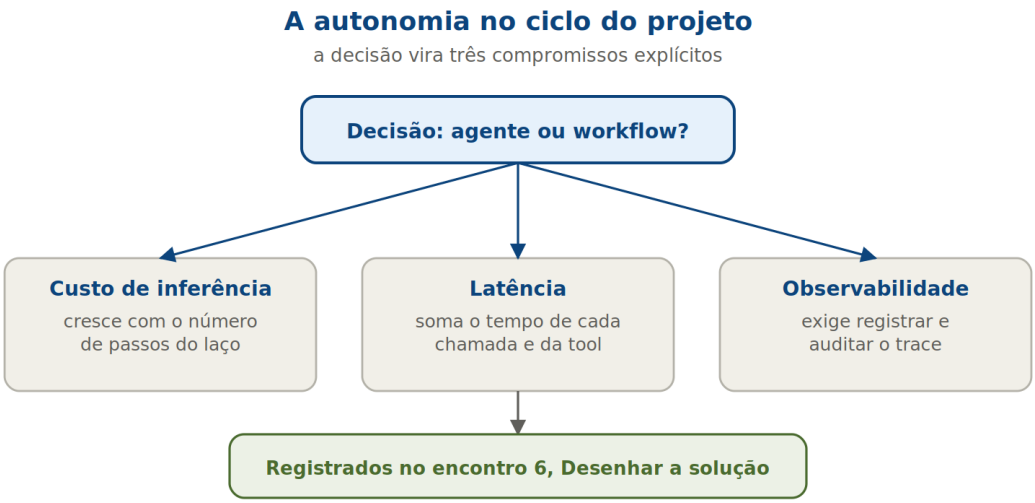
GLOSSÁRIO

**observabilidade**

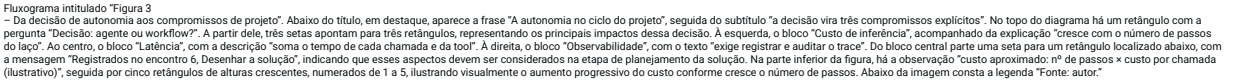
Capacidade de inspecionar, registrar e auditar o comportamento interno de um sistema em execução. Para um agente, significa ter o trace de raciocínio, ação e observação de cada passo disponível para depuração e para auditoria, e não apenas o resultado final.

O terceiro compromisso é a observabilidade, que precisa ser instrumentada desde o início, e não acrescentada depois que um problema aparece. Sem o registro do raciocínio, da ação e da observação a cada passo, não há como auditar nem depurar o agente quando o resultado vem errado, porque o que se vê de fora é apenas a saída final, sem o histórico que a produziu. Em setor regulado, a observabilidade deixa de ser conveniência de engenharia e vira requisito, porque é preciso poder reconstruir e justificar cada decisão automatizada perante um auditor ou um regulador. Instrumentar o trace é, portanto, um custo de início de projeto, não um acréscimo opcional.

Figura 3 - Da decisão de autonomia aos compromissos de projeto




Fonte: autor.


Fluxograma intitulado "Figura 3
– Da decisão de autonomia aos compromissos de projeto". Abaixo do título, em destaque, aparece a frase "A autonomia no ciclo do projeto", seguida do subtítulo "a decisão vira três compromissos explícitos". No topo do diagrama há um retângulo com a pergunta "Decisão: agente ou workflow?". A partir dele, três setas apontam para três retângulos, representando os principais impactos dessa decisão. À esquerda, o bloco "Custo de inferência", acompanhado da explicação "cresce com o número de passos do laço". Ao centro, o bloco "Latência", com a descrição "soma o tempo de cada chamada e da tool". À direita, o bloco "Observabilidade", com o texto "exige registrar e auditar o trace". Do bloco central parte uma seta para um retângulo localizado abaixo, com a mensagem "Registrados no encontro 6, Desenhar a solução", indicando que esses aspectos devem ser considerados na etapa de planejamento da solução. Na parte inferior da figura, há a observação "custo aproximado: nº de passos × custo por chamada (ilustrativo)", seguida por cinco retângulos de alturas crescentes, numerados de 1 a 5, ilustrando visualmente o aumento progressivo do custo conforme cresce o número de passos. Abaixo da imagem consta a legenda "Fonte: autor."

Para a estimativa de ordem de grandeza, vale fixar uma regra prática que pode ser aplicada já na fase de proposta. O custo de um agente é da ordem do número de passos vezes o custo por chamada ao modelo, somado ao custo de execução das tools. Compare esse número com o custo de uma chamada direta, que é uma unidade de referência. Se o agente típico usa quatro passos e o pior caso usa oito, dimensione pelo pior caso, ou imponha um teto de iterações que torne o custo máximo previsível e financiável. A latência segue exatamente o mesmo raciocínio: estime pelo caminho mais longo, e não pelo mais curto, porque é o caminho mais longo que o usuário vai eventualmente encontrar.

A razão para dimensionar pelo pior caso, e não pela média, é a própria natureza do agente. Como o caminho é decidido em execução, ele varia, e a variação é justamente o que justificou a escolha do agente no tópico 1. Um sistema cujo caminho varia não tem um custo único, tem uma distribuição de custos, e orçar pela média ignora a cauda dessa distribuição, que é onde moram os casos caros. Impor um teto de iterações é a forma mais simples de cortar essa cauda: troca-se uma pequena perda de capacidade nos casos extremos por um custo máximo conhecido, o que costuma ser um bom negócio em ambiente de produção.

### Exemplo aplicado

Ao dimensionar o agente de triagem de chamados, a equipe estimou quatro passos no caminho típico e oito no pior caso, contra uma única chamada na versão workflow. Concluiu que, no volume diário previsto, o agente custaria várias vezes mais em inferência e responderia mais devagar. A decisão foi impor um teto de seis iterações e restringir o agente à fila de exceções, mantendo o workflow no caminho comum. O teto tornou o custo de pior caso previsível e financiável.

Aqui se resolve a última pergunta reflexiva do notebook da etapa passada. Naquele notebook, a tarefa A levou dois passos e a tarefa B levou três, sendo um deles um erro com recuperação. Num agente real, cada passo é uma chamada ao modelo, com seu custo e sua latência. Estimar o custo do sistema pela tarefa A subestimaria o que ele realmente consome. O dimensionamento honesto parte do pior caso observado, porque é a variação de caminho, e não a média, que define o teto de custo e de latência. Foi por isso que a tarefa B, com seu passo extra de recuperação, era a mais informativa para quem precisa orçar a solução, e não apenas para quem quer ver o agente funcionar.

No ciclo da Formação 1, esses três compromissos se registram no encontro 6, Desenhar a solução. A decisão entre agente e workflow não pode ficar implícita na arquitetu-

ra: ela precisa ser declarada, com sua justificativa e com suas consequências de custo, latência e observabilidade. Os dois instrumentos do módulo servem exatamente a isso. O template T2.1, matriz de decisão workflow versus agente, organiza a escolha pelos três eixos do tópico 1. O template T2.2, matriz de modos de falha e mitigações, registra os riscos do tópico 2 e o que será feito a respeito de cada um. Você preencherá ambos a seguir, aplicados a um cenário industrial dado, e é esse preenchimento, e não a leitura do texto, que constitui a evidência de aprendizagem do módulo.

**ATENÇÃO**

A confusão decisória de fechamento é orçar o agente pelo caminho feliz e deixar a observabilidade para depois. O resultado típico aparece em produção: o custo real supera o previsto no pior caso, e, quando uma decisão precisa ser auditada, não há trace para consultar. Custo e observabilidade são compromissos de início de projeto, não ajustes finais.

**SAIBA MAIS**

O aprofundamento técnico da operação de agentes, incluindo design avançado de tools, multi-agente em produção, evaluation de traces e mecanismos de fallback, é tratado no curso de LLMs da Fase 2 do programa.

A lente da autonomia se resume, no fim, a duas decisões encadeadas: quem controla o caminho, o sistema ou quem o desenhou, e quanto custa sustentar essa escolha com qualidade ao longo do tempo. Decidir bem essas duas questões, antes de implementar e com os números na mesa, é o que separa um uso maduro de IA Agentic de uma adoção guiada por modismo.

## 4. Conclusão

A camada decisória do módulo M2 organizou a lente da autonomia em torno de três decisões encadeadas. A primeira é a heurística workflow versus agente, que coloca o workflow como default e reserva o agente para o branching dinâmico irrelutável que agrega valor real. A segunda é o tratamento dos modos de falha recorrentes, loops, seleção de tool errada, prompt injection e cascading errors, com mitigações que agem

entre a observação e a próxima decisão. A terceira é a tradução da escolha em compromissos de projeto: custo de inferência, latência e observabilidade, dimensionados pelo pior caso.

O fio que une as três decisões é uma inversão do ônus da prova. Em IA Agentic, quem precisa se justificar é a autonomia, e não o workflow. Essa inversão protege o projeto do over-engineering agentic, em que se paga custo, latência e imprevisibilidade sem um ganho proporcional. A maturidade no uso da lente está menos em saber montar um agente e mais em decidir, com critério e com números, se ele é necessário.

Os instrumentos que sustentam essas decisões são os templates T2.1, matriz de decisão workflow versus agente, e T2.2, matriz de modos de falha e mitigações. Eles conectam o módulo ao encontro 6 da Formação 1, Desenhar a solução, em que a opção por um agente precisa ser registrada com seus compromissos explícitos antes de a arquitetura ser fechada. O preenchimento desses instrumentos é a evidência de aprendizagem da camada decisória.

Como ângulo de leitura, a autonomia atravessa os sete eixos do programa. Ela é eixo natural de LLMs e NLP, mas aparece também em Engenharia de Dados, com agentes de inferência de schema, em Real Time Processing, com agentes assíncronos, e em Visão Computacional, com agentes que orquestram tools de percepção. Em todos os casos, a decisão de fundo é a mesma: quem controla o caminho, e a que custo.

REFERÊNCIAS

## Bibliografia mínima

YAO, S. et al. ReAct: Synergizing Reasoning and Acting in Language Models. ICLR, 2023. Preprint em https://arXiv.org/abs/2210.03629.

SCHICK, T. et al. Toolformer: Language Models Can Teach Themselves to Use Tools. NeurIPS, 2023. Preprint em https://arXiv.org/abs/2302.04761.

ANTHROPIC. Building Effective Agents. Anthropic Engineering, 2024. Disponível em https://anthropic.com/engineering/building-effective-agents.

ANTHROPIC. Model Context Protocol Specification. https://modelcontextprotocol.io, edição em curso.

## Bibliografia complementar

WANG, L. et al. A Survey on Large Language Model based Autonomous Agents. Frontiers of Computer Science, 2024. Preprint em https://arXiv.org/abs/2308.11432.

SHINN, N. et al. Reflexion: Language Agents with Verbal Reinforcement Learning. NeurIPS, 2023. Preprint em https://arXiv.org/abs/2303.11366.

LANGCHAIN. LangGraph Documentation. https://langchain-ai.github.io/langgraph, edição em curso.

WU, Q. et al. AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation. arXiv:2308.08155, 2023.

# Industr.IA

Residência em TIC 49



Executor iRede

Executor Atlântico

Coordenador Softex

Financiador MCTI Futuro

Ministério da Ciência, Tecnologia e Inovação

Governo do Brasil — Do lado do povo brasileiro

Projeto apoiado pelo Ministério da Ciência, Tecnologia e Inovações, com recursos da Lei nº 8.248, de 23 de outubro de 1991.